→ What is RAG?

To understand RAG, we will work with the example of a company that wants to deploy a chatbot as an intelligent knowledge assistant. For the chatbot to be useful, it will need to be connected to the company's knowledge base.

Connecting the chat endpoint with external knowledge

This allows the chatbot to have the correct context when responding to requests, such as summarizing a meeting transcript, extracting info from the company wiki, and assisting a customer support agent in responding to a customer inquiry. Without access to the company's knowledge base, the chatbot will not be able to perform these type of tasks successfully.

Without RAG

Where did the last AI conference take place?

New York

[X]

With RAG

Where did the last AI conference take place?

"The 2023 AI conference in London gathered thousands of ---."

London

[✓]

RAG solves the lack of specific knowledge probm.

The company will
respond to time-s
answers. For ex.,
about a recent p
with data that
cut-off time.
model relies on
been trained on
in the training
recency in th
inadequate an

Without RAG

How man
do ABC

I'm s
have
ABC

↳ RAG consists
augmented

at how

⇒ What is RAG?

To understand RAG, we will work with the example of a company that wants to deploy a chatbot as an intelligent knowledge assistant. For the chatbot to be useful, it will need to be connected to the company's knowledge base.

Connecting the chat endpoint with external knowledge.

This allows the chatbot to have the correct context when responding to requests, such as summarizing a meeting transcript, extracting info from the company wiki, and assisting a customer support agent in responding to a customer inquiry. Without access to the company's knowledge base, the chatbot will not be able to perform these type of tasks successfully.

Without RAG

Where did the last AI conference take place?

New York ☒

With RAG

Where did the last AI conference take place?

"The 2023 AI conference in London gathered thousands of ---."

London ✓

RAG solves the lack of specific knowledge probm.

The company will a
respond to time-sens
answers. For ex., sup
about a recent publ
with data that is
cut-off time. Wit
model relies on th
been trained on (
in the training da
recency in the
inadequate answ

Without RAG

How many e
do ABC Co

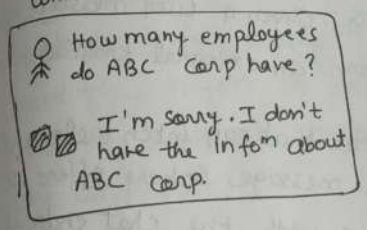
I'm sorr
have the
ABC Corp

↳ RAG consists
augmented g

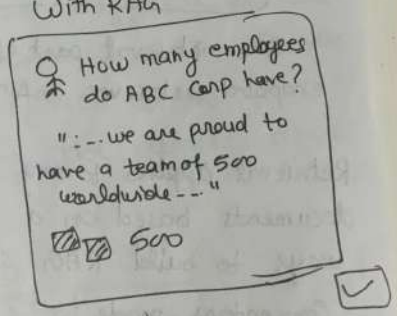
at how th

The company will also likely need the chatbot to respond to time-sensitive prompts & provide up-to-date answers. For ex., suppose an employee asks the chatbot about a recent public event. A baseline LLM is trained with data that is ~~currently~~ current up to a certain cut-off time. Without accessing external data, the model relies on the most recent information it has been trained on (assuming the specific info^m is available in the training data). In this situation, the lack of recency in the training data would produce an inadequate answer.

Without RAG



With RAG

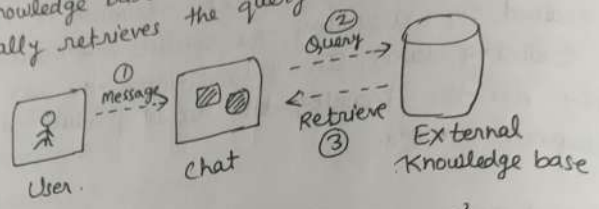


RAG solves the lack of recency prob^m.

↳ RAG consists of 2 parts: a retrieval system and an augmented generation system. Let's take a look at how they work.

⇒ Retrieval

The first part of RAG is to retrieve the right inform needed to respond to a user query. Given a user message (1), the chat endpoint queries an external knowledge base with the relevant queries (2) and finally retrieves the query results (3).



The retrieval part of RAG: Given a user message, the endpoint retrieves inform from an external knowledge base.

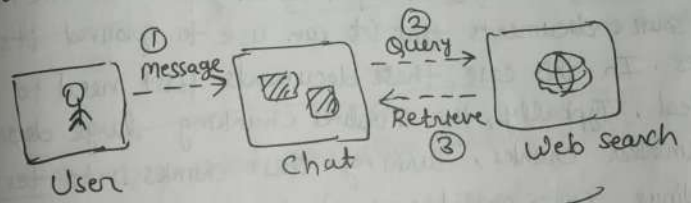
Retrieval applies to any system that can fetch relevant documents based on a user message. Cohere offers 2 ways to build RAG systems with the chat endpoint: Connectors mode & documents mode.

↳ Connectors Mode

As the name implies, connectors are ways of connecting to data sources. These data sources could be internal documents, document databases, the broader internet, or any other data source which can inform the model's replies. When the Chat endpoint is called in connectors mode, it connects to the associated data sources & automatically handles document retrieval.

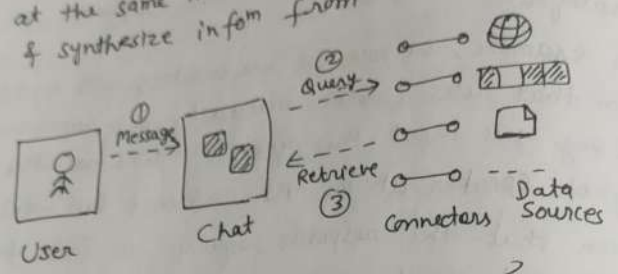
Developers can leverage pre-built connectors to various data sources or even their own. Also, there is a web search connector managed by Cohere which runs searches against a browser in safe mode. Developers can use it immediately w/o any additional configuration or deployment.

For example, suppose we are working with a RAG system that uses Cohere's managed web search connector, and say you input this query: "Who was the keynote speaker at the AI conference last week?". Given that the response requires a fact from a recent event, the chatbot triggers a retrieval of this info using the web search API. It gets back the info it requires, such as a few website snippets containing the details about the conference.



An example of retrieving info via web search.

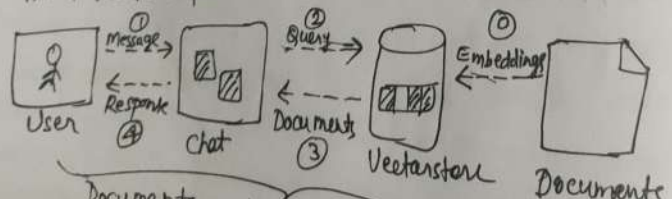
In an enterprise setting, data is distributed across multiple platforms & datastores, With connectors mode, you can use multiple connectors at the same time to get the system to retrieve & synthesize info from various data sources.



Connectors mode allows you to use multiple connectors to connect to multiple datastores.

↳ Documents Mode

With documents mode, a developer directly provides the model with source documents that it can use to ground its replies. In this case, these documents first need to be ingested. Typically, this involves chunking large documents into smaller chunks, turning these chunks into text embeddings (also called vectors) & storing these embeddings in a vector ~~space~~ store.



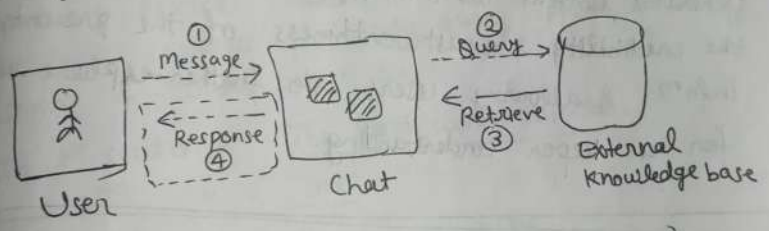
Documents are ingested before retrieval.

Connectors mode automatically handles document ingestion is excellent for those who prefer a more hands-off approach. However, documents mode allows developers to customize the RAG stack's components to fit specific needs.

Augmented Generation

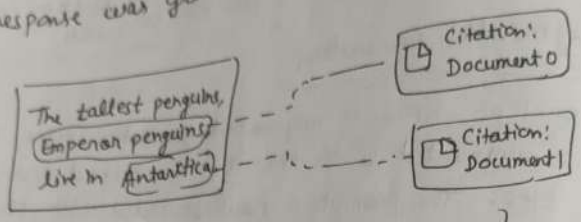
The 2nd part of RAG is augmented generation. Here, the prompt is augmented with the infom retrieved from the retrieval step. The prompt is now grounded with the best infom to provide the user with an accurate & helpful response.

The chatbot responds to the user query, now having the augmented prompt as its context.



The augmented generation part of RAG: The Chat endpoint uses the retrieved infom to provide a grounded response.

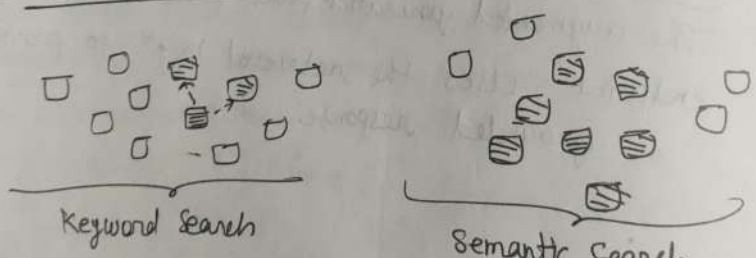
Cohere's Chat endpoint also provides citations to indicate the parts of the retrieved documents on which the response was grounded.



Citations provide verifiable references for the generated content.

↳ Citations provide a critical benefit by delivering the generated content with verifiable references, enhancing the credibility & trustworthiness of the presented info^n & allowing users to further explore responses for a deeper understanding.

✱ Semantic Search



tions to indicate
which the

to

:
+1

the

ng the
nhancing
esented
re responses

Here, you'll learn how to use embeddings & similarity in order to build a semantic search model.

Semantic search is a powerful way to search in a large databases using the context of the query

→ What is not semantic search? (Lexical search)

Lexical search
(word matching)

Ex: Query: Where was the last world cup?

↳ It is basically a word matching.

→ The idea is to search in the data set for the answer or atleast the closest things to an answer, So let's say that the positive answers are: The last world cup was in Qatar, The sky is blue, The bear lives in woods, an apple is a fruit.

Sentences
The last world cup was in Qatar
The sky is blue
The bear lives in the woods
An apple is a fruit

↳ The way lexical search works is by word matching. So, it looks words from the query appear in the answer & counts them. So,

In 1st sentence: The last world cup was

In 2nd " : The

In 3rd " : The ---- the

In 4th " : [Nothing is matching]

So, count in 1st sentence = 5 ⇒ Count is highest & so, this time it did okay & actually found the answer.

" " 2nd " = 1

" " 3rd " = 1

" " " " = 0

↳ Prob^{m}s in Lexical Search

Ex.

Query: where was the last world cup?

Now, here, matching words are:

In 1st sentence: The .. world cup was - -
" 2nd " : The cup .. where - - -
" 3rd " : where - - - the world - - - - - - last cup
" 4th " : [Nothing is matching]

Sentences
The previous world cup was in Qatar
The cup is where you left it
Where in the world is my last cup of coffee?
An apple is a fruit

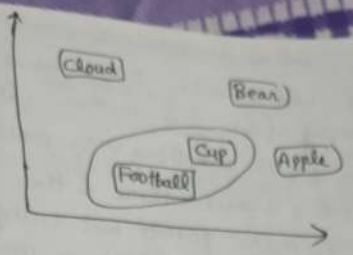
So, Count in 1st sentence : 4
" " 2nd " : 3
" " 3rd " : 5 ⇒ It wins even though it is not the answer.
" " 4th " : 0

⇒ So, counting words in common is not the ideal way to search. So, we don't want lexical search, Instead, we want Semantic Search where instead of words, the actual meaning of the sentence is taken into account. & for that we need embeddings

⇒ How to search using a text embedding?

We are going to use nearest neighbours.

Sentence	Say, Sentence is represented by
The last world cup was in Qatar	→ Football
The sky is blue	→ Cloud
The bear lives in woods	→ Bear
An apple is a fruit	→ Apple



Query: Where was the last world cup? ⇒ Cup

"Cup" is close to "Football" because semantically the embedding know that these 2 sentences are similar, so the winner is: "The last world cup was in Qatar".

As you can see, this way of searching actually takes into account the meaning of the sentence & not just the words. So, it is more effective.

[Check on Cohere Playground ⇒ 'Embed' tab]

⇒ Prob^m with Nearest Neighbors: Good but slow

↳ We have to calculate lots of distances | similarity. If we have 8 sentences, then we have to find 8^2 distances. So, for n sentences, n^2 distances.

↳ Some solutions to this

- Inverted file index (IVD): Cluster, then search
- Hierarchical Navigable Small Worlds (HNSW): Start with less points, search there. Then add more & iterate.